

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

Total Marks: 30

Code of Conduct

I Samiksha Dhanya G(192321014) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Samiksha Dhanya G

Assessment Tasks

Case Study

1. Case Study 1: Smart Retail Inventory Management System

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

2. Case Study 2: Cloud-Based Document Collaboration System

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices. Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

3. Case Study 3: Online Food Ordering Platform

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

1. Case Study: Smart Retail Inventory Management System

A Smart Retail Inventory Management System is a cloud-based solution designed to help retail chains manage products, stock levels, sales transactions, and demand forecasting efficiently. Traditional inventory systems often store data separately at individual stores, making it difficult to obtain real-time information across the entire retail chain. By moving the system to the cloud, all stores can connect to centralized services and access updated information whenever required.

The main clients of the system include Point of Sale (POS) systems, store computers, mobile devices, and management dashboards. These clients communicate with centralized cloud services through Application Programming Interfaces (APIs). For example, whenever a product is sold through a POS system, the transaction is sent through an API to the cloud server. The inventory database is then updated automatically, reducing the possibility of manual errors and outdated stock information.

Cloud storage plays an important role in maintaining different types of data, including product information, transaction logs, supplier details, customer data, and analytics datasets. Since the data is centrally stored, managers can monitor inventory across multiple store locations through a single dashboard. This also makes backup, recovery, and data management easier compared with maintaining separate local databases.

The platform also uses cloud-based analytics and demand forecasting services. Historical sales data can be analysed to identify frequently purchased products, seasonal demand, and changing customer behaviour. Based on these patterns, the forecasting system can predict future inventory requirements. For instance, if the system identifies increased sales of a particular product during a festival season, it can recommend that stores maintain higher stock levels before the next season. This helps prevent both overstocking and stock shortages.

Thus, the Smart Retail Inventory Management System demonstrates the integration of cloud infrastructure, storage, APIs, analytics services, web applications, and security mechanisms. The cloud architecture improves scalability, provides real-time inventory visibility, supports intelligent decision-making, and allows the retail chain to manage operations more efficiently.

2. Case Study: Cloud-Based Document Collaboration System

A Cloud-Based Document Collaboration System is designed to allow multiple users to create, edit, share, and manage documents through the internet. A common example of this type of system is an online document editor where several users can work on the same file simultaneously. Unlike traditional desktop applications, users do not need to install or maintain separate copies of documents on every device.

Users access the system through web browsers on computers, tablets, or mobile devices. After authentication, they can create new documents or access existing shared files. The web application provides the user interface for editing, formatting, commenting, and sharing documents. Since the application is cloud-based, users can access their documents from different devices and locations as long as they have the necessary permissions and an internet connection.

Documents are stored in distributed cloud storage. Distributed storage improves availability and reliability by maintaining data across multiple servers or locations. If one server experiences a failure, the data can

still be available through another location. This architecture also supports large numbers of users and documents without depending on a single physical storage system.

Version control is another important feature of the collaboration system. Each major change to a document can be recorded, allowing users to view previous versions and restore an earlier copy if required. This prevents the permanent loss of important information when an incorrect modification is made. It also allows teams to track how a document has changed over time.

Overall, the Cloud-Based Document Collaboration System demonstrates how web applications, cloud storage, APIs, distributed infrastructure, and security services work together. It provides real-time collaboration, accessibility, reliability, and secure document management for individuals and organizations.

3. Case Study: Online Food Ordering Platform

An Online Food Ordering Platform is a cloud-based application that allows customers to browse restaurants, view menus, place orders, make payments, and track deliveries through web browsers or mobile applications. The system connects customers, restaurants, delivery services, and administrators through a centralized cloud environment.

The frontend acts as the interface between users and the application. Customers use a web or mobile client to search for restaurants, select food items, add products to a cart, and place an order. Restaurant owners can use another interface to manage menus and receive incoming orders. These frontend applications communicate with backend services through RESTful Web APIs.

The backend processes important operations such as user registration, authentication, order management, payment processing, restaurant management, and delivery updates. REST APIs allow the frontend and backend to exchange information in a structured manner. For example, when a customer places an order, the frontend sends an API request to the backend. The backend validates the order, stores the required information, and notifies the restaurant.

Cloud storage is used for menu images, invoices, customer information, order records, and other application data. Secure cloud storage provides durability and easy access while reducing the need for the startup to maintain its own physical storage infrastructure. Backups and access controls can also be configured to protect important business and customer information.

The infrastructure uses Virtual Machines (VMs) and managed Kubernetes services. Virtual machines provide the computing resources required to run backend applications and supporting services. Kubernetes manages containerized application components and helps the platform scale according to demand. During busy periods, such as lunch or dinner hours, additional application instances can be created automatically to handle increased traffic. This improves performance and reliability.

In conclusion, the Online Food Ordering Platform demonstrates the real-time integration of frontend clients, RESTful APIs, cloud storage, virtual infrastructure, Kubernetes, and security mechanisms. The cloud-based architecture provides scalability, reliability, accessibility, and efficient management, allowing the startup to support a growing number of users and transactions.